SmartPlanif | Audit des algorithmes et de l’architecture

- Allocation quotidienne bus / chauffeur / receveur
- Lecture critique du code fourni, sous l’angle recherche opérationnelle
- Architecture, heuristiques, ALNS, garanties et risques
- Périmètre : audit statique ; aucune exécution .NET ou MySQL réalisée

Source : README.md ; AllocateDay.cs

Synthèse : une approche hybride pertinente

- Construction rapide, puis amélioration par ALNS et recuit simulé
- Séparation claire entre faisabilité dure et préférences souples
- Architecture modulaire et calcul en mémoire
- Limites majeures : sélection des agents, objectif approximatif, cohérence du diagnostic

Source : ConstructionStrategies.cs ; AlnsImprover.cs ; ObjectiveFunction.cs

Quel problème est réellement résolu ?

- Les voyages et leurs horaires sont des données d’entrée
- Chaque voyage reçoit un bus et deux agents distincts
- Les ressources sont réutilisables si leurs plannings le permettent
- Ce n’est pas un calcul complet d’itinéraires ou d’horaires

Source : Entities.cs ; TripAllocator.cs ; AllocateDay.cs

Formalisation proposée : variables et couverture

- x[t,b] = 1 si le bus b réalise le voyage t
- y[t,a] et z[t,a] = affectations chauffeur et receveur
- u[t] = 1 si le voyage reste non affecté
- Pour chaque t : somme x = somme y = somme z = 1 − u[t]
- Pour chaque agent a : y[t,a] + z[t,a] ≤ 1

Source : Modélisation analytique à partir de TripAllocator.cs

Contraintes : les catégories ne sont pas interchangeables

- Verrouillées dures : bus actif, absence de chevauchement bus et agent
- Dures désactivables : repos, heures, amplitude, maintenance, kilomètres
- Dures ou souples : centre, délégation, catégorie compatible
- Préférences pures : continuité, charge faible, âge du bus
- La conformité dépend des règles effectivement activées

Source : ConstraintCatalog.cs

Architecture : responsabilités et dépendances

- Web : composants Blazor en mode Interactive Server
- Application : cas d’usage et interfaces de persistance
- Domain : entités, règles, plannings, heuristiques et ALNS
- Infrastructure : MySQL / Dapper, démonstration, paramètres JSON
- Application → Domain ; Infrastructure → Application ; Web compose les services

Source : Program.cs ; Ports.cs ; fichiers .csproj

Flux d’une allocation de bout en bout

- 1. Contrôler la date et charger la politique
- 2. Charger voyages, ressources et affectations sur J−1 à J+1
- 3. Distinguer voyages verrouillés et voyages à affecter
- 4. Construire, améliorer, compléter, puis contrôler
- 5. Enregistrer seulement si les garde-fous le permettent

Source : AllocateDay.cs ; PlanningOptions.cs

État partagé et planification en mémoire

- ScheduleBook conserve les plannings des bus et des agents
- Le même planning d’agent couvre ses deux fonctions
- Assign / Unassign modifient simultanément les occupations
- Clone isole un candidat ALNS du plan courant
- Les affectations verrouillées occupent déjà les ressources

Source : ScheduleBook.cs ; ResourceSchedule.cs ; AllocationState.cs

Phase 0 : diagnostic de capacité

- Pic de simultanéité obtenu par balayage des événements
- Les fins passent avant les débuts en cas d’égalité
- Comparaisons : bus globaux, bus compatibles par type, agents, heures
- Un résultat suffisant n’établit pas la faisabilité globale
- Le diagnostic ne soustrait pas explicitement toutes les occupations verrouillées

Source : FeasibilityAnalyzer.cs

Diagnostic : incohérences à corriger

- Le filtre du jour de repos est appliqué même si A03 est désactivée
- Les pics incluent les marges de repos même si A07 / B09 sont désactivées
- Le budget d’heures utilise MaxDailyWork même si A08 est désactivée
- Risque : annoncer une insuffisance incompatible avec la politique réelle
- Partager les règles actives entre diagnostic et allocation

Source : FeasibilityAnalyzer.cs ; ConstraintCatalog.cs

Heuristique 1 : glouton chronologique

- Trier les voyages par départ puis identifiant
- Pour chaque voyage : filtrer, noter et choisir les ressources
- Conserver immédiatement l’affectation réalisable
- Atout : simplicité, référence de comparaison, rapidité potentielle
- Faiblesse : un choix local peut consommer une ressource rare

Source : ConstructionStrategies.cs ; TripAllocator.cs

Heuristique 2 : construction par tension

- Regrouper les voyages par type de ligne
- Tension = pic de demande / max(1, bus compatibles)
- Traiter les types par tension décroissante
- Dans chaque type : traiter les voyages avec le moins d’options
- Tous les types partagent le même état de ressources

Source : ConstructionStrategies.cs

Difficulté et sélection des candidats

- Difficulté = min(bus éligibles, options d’agents)
- Options agents = min(chauffeurs, receveurs, union des agents / 2)
- Rejeter au premier échec de règle dure
- Score souple = somme des poids × valeurs normalisées entre 0 et 1
- Classement : score + bruit éventuel, puis départage déterministe

Source : TripAllocator.cs ; CandidateSelector.cs

Défaut critique : la paire d’agents n’est pas optimisée

- Le code choisit d’abord le meilleur chauffeur
- Puis il retient le premier receveur différent de ce chauffeur
- Contre-exemple : A polyvalent classé premier ; B chauffeur seulement
- Le code choisit A chauffeur et ne trouve plus de receveur
- La paire B chauffeur / A receveur est pourtant réalisable

Source : TripAllocator.TryAssign ; exemple analytique

Fonction objectif effectivement codée

- Minimiser $F = P \times U + D + \lambda a \times \sigma a + \lambda b \times \sigma b$
- U : voyages non affectés ; D : déficit de score souple
- σa et σb : écarts-types des heures agents et bus
- Valeurs du code : P = 100 000 ; λa = λb = 10
- Les heures d’équité couvrent tout le ScheduleBook chargé

Source : ObjectiveFunction.cs ; AllocationPolicy.cs

L’objectif n’est pas une hiérarchie garantie

- Le validateur vérifie P > 0, pas sa dominance sur les autres termes
- Des poids configurables peuvent renverser la priorité de couverture
- Les scores mémorisés proviennent des contextes d’insertion
- Le même plan peut recevoir des valeurs différentes selon son historique
- Proposition : comparaison lexicographique et réévaluation cohérente

Source : ObjectiveFunction.cs ; PolicyValidator.cs ; PlanEvaluator.cs

ALNS : explorer de grands voisinages

- Copier le plan courant
- Choisir une destruction et une réparation selon leurs poids
- Retirer une fraction des affectations puis réinsérer les voyages en attente
- Accepter ou rejeter le candidat ; conserver séparément le meilleur
- Arrêter sur itérations, durée, stagnation ou annulation

Source : AlnsImprover.cs

Quatre destructions, trois réparations

- Destructions : aléatoire ; pires scores ; voisinage temporel ; ressource
- Le pivot temporel cible un voyage non affecté avec probabilité 0,7 lorsqu’il en existe
- Réparations : chronologique ; plus contraint ; ordre aléatoire
- Les deux dernières peuvent bruiter le classement des ressources
- Le retrait par ressource distingue le poste chauffeur du poste receveur

Source : Operators.cs

Recuit simulé et adaptation des opérateurs

- Si $\Delta = F(\text{candidat}) - F(\text{courant}) > 0$: acceptation $\exp(-\Delta / T)$
- Refroidissement géométrique : $T \leftarrow 0{,}998 \times T$ par défaut
- Récompenses : nouveau meilleur 33 ; amélioration 9 ; accepté 3
- Poids $\leftarrow \max(0{,}1 ; (1-\rho) \times \text{poids} + \rho \times \text{récompense})$
- Destruction et réparation reçoivent la même récompense

Source : AlnsImprover.cs ; AllocationPolicy.cs

Réglages : ce qu’ils signifient réellement

- 3 000 itérations ; 20 secondes ALNS ; stagnation 1 500
- Destruction 5–15 %, mais au moins deux affectations si possible
- Température initiale 100 ; bruit 10 % ; graine 42
- P = 100 000 rend très improbable la perte temporaire d’un voyage à T = 100
- La graine seule ne garantit pas le même résultat si l’arrêt temporel intervient

Source : AllocationPolicy.cs ; AlnsImprover.cs

Complétion et validation : garanties limitées mais utiles

- La complétion retente les voyages restants sans déplacer les affectés
- Le contrôle retire temporairement un voyage puis rejoue ses règles dures
- Il vérifie aussi les qualifications et la distinction des agents
- Il réutilise les mêmes implémentations de règles que la construction
- Dans AllocateDay, les affectations verrouillées ne sont pas toutes auditées isolément

Source : PlanCompleter.cs ; PlanValidator.cs ; AllocateDay.cs

Persistance et exploitation

- Simulation sans écriture ; compléter ou remplacer le plan
- Écriture transactionnelle des seules colonnes prévues *_PR
- Protection des voyages aux trois ressources réelles renseignées
- Refus si violations ou journée incomplète dans le mode prévu
- Atomicité d’écriture ≠ protection contre un calcul devenu obsolète

Source : AllocateDay.cs ; MySqlAssignmentRepository.cs

Limites métier et qualité des données

- Continuité de station : préférence souple, pas contrainte de trajet à vide
- Pas de matrice explicite des temps de transfert dans les règles inspectées
- Capacité contrôlée seulement si ExpectedPassengers est connu
- Durées futures parfois estimées depuis l’itinéraire de ligne
- Polyvalence par défaut et activation automatique des bus à valider

Source : SoftRules.cs ; BusRules.cs ; README.md

Forces et faiblesses de l’architecture

- Forces : domaine autonome, ports explicites, règles extensibles, mode démo
- Forces : calcul en mémoire, explications, progression et sauvegarde séparée
- Faiblesses : listes parcourues souvent, clones répétés et tris à chaque ajout
- Risques : paramètres JSON et calculs concurrents à gouverner
- Program.cs ne montre pas de configuration explicite authentification / autorisation

Source : ResourceSchedule.cs ; Program.cs ; Ports.cs ; AllocateDay.cs

Performance et niveau de preuve

- Aucun benchmark exécuté dans cet audit
- README : 34/36 glouton ; 35/36 phases ; 36/36 ALNS ; zéro violation annoncé
- README : 58 tests annoncés avec lanceur de remplacement
- Dapper / MySqlConnector réels et interactions navigateur non validés selon le README
- Un cas de démonstration ne prouve ni supériorité générale ni passage à l’échelle

Source : README.md ; ResourceSchedule.cs

Protocole scientifique recommandé

- Même jeu de données, politique, verrouillages et budget pour chaque méthode
- Petites instances : modèle exact de référence et borne de qualité
- Instances réalistes : journées ordinaires, pics, pénuries, absences, passages de minuit
- Plusieurs graines ; médiane, dispersion, pire cas et temps de calcul
- Mesurer couverture, violations, qualité exacte, équité et robustesse aux perturbations

Source : Recommandations issues des limites observées

Feuille de route priorisée

- P0 : corriger la sélection de paire et aligner le diagnostic sur les règles actives
- P0 : rendre la priorité de couverture explicite ; aligner score recherché et score publié
- P0 : tester les vraies dépendances et sécuriser les écritures concurrentes
- P1 : temps de transfert, qualifications, contrôle des verrouillés, équité adaptée aux disponibilités
- P2 : regret-k, échanges ciblés, profilage, calcul incrémental, référence exacte

Source : Synthèse de l’audit statique

Conclusion et traçabilité

- Bon socle modulaire pour une aide à la décision explicable
- ALNS apporte une capacité de remise en cause des choix gloutons
- Le code ne fournit pas de certificat d’optimalité
- La priorité est la cohérence du modèle avant l’ajout de métaheuristiques
- Source unique : archive texte du dépôt SmartPlanif fournie par l’utilisateur